

Model Development Phase Template

Date	21 June 2025
Team ID	SWTXT_20251747173488
Project Title	Productivity Prediction Web App
Maximum Marks	4 Marks

Model Training, Validation, and Evaluation Report: Productivity Prediction Web App

Saurabh Yadav

21 June 2025

Introduction

The Productivity Prediction Web App forecasts garment worker productivity using machine learning models trained on the "Productivity Prediction of Garment Employees" dataset (1197 records, 14 input features). This report documents the initial training code for Linear Regression, Random Forest, and XGBoost models, and evaluates their performance using regression metrics: R^2 Score, Mean Absolute Error (MAE), and Mean Squared Error (MSE). The Random Forest model achieved the highest R^2 score (0.547), followed by XGBoost (0.482) and Linear Regression (0.197), guiding the app's predictive capabilities.

Initial Model Training Code

The following Python code trains the three models on the preprocessed dataset. The code handles data loading, preprocessing, splitting, model training, and saving for deployment. A screenshot of this code should be inserted below.

```
# train_models.py

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
import joblib

# Load dataset
data = pd.read_csv('garments_worker_productivity.csv')

# Preprocessing
data['month'] = pd.to_datetime(data['date']).dt.month
data['wip'] = data['wip'].fillna(data['wip'].mean())
data['department'] = data['department'].replace('sweing', 'sewing')
for col in ['quarter', 'department', 'day', 'team']:
    data[col] = LabelEncoder().fit_transform(data[col])

# Features and target
X = data.drop(['date', 'actual_productivity'], axis=1)
```

```
y = data['actual_productivity']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize models
lr_model = LinearRegression()
rf_model = RandomForestRegressor(random_state=42)
xgb_model = XGBRegressor(random_state=42)

# Train models
lr_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)
xgb_model.fit(X_train, y_train)

# Save models
joblib.dump(lr_model, 'lr_model.sav')
joblib.dump(rf_model, 'rf_model.sav')
joblib.dump(xgb_model, 'xgb_model.sav')
```

Instructions: Copy the above code into a Python file (e.g., `train\_models.py`), run it in an IDE (e.g., VSCode), and take a screenshot of the code editor displaying the code. Insert the screenshot below this section.

Model Validation and Evaluation Report

The models were validated on a 20% test set (240 records) using regression metrics: R^2 Score (proportion of variance explained), MAE (average prediction error), and MSE (squared error). Results are summarized below, with Random Forest outperforming others.

Model	R^2 Score	Mean Absolute Error (MAE)	Mean Squared Error (MSE)
Linear Regression	0.197	0.107	0.021
Random Forest	0.547	0.069	0.012
XGBoost	0.482	0.071	0.014

Evaluation Details:

- Linear Regression: Low R^2 (0.197) indicates poor fit, likely due to non-linear relationships in the data. Higher MAE (0.107) and MSE (0.021) suggest limited predictive accuracy.
- Random Forest: Highest R^2 (0.547) reflects strong performance, capturing 54.7% of variance. Lowest MAE (0.069) and MSE (0.012) indicate precise predictions, ideal for the app.
- XGBoost: Moderate R^2 (0.482) and slightly higher MAE (0.071) and MSE (0.014) than Random Forest, but still effective for complex patterns.

Validation Approach:

- Data Split: 80% training (957 records), 20% testing (240 records) with `random_state=42` for reproducibility.
- Metrics Calculation: Used Scikit-learn's `r2_score`, `mean_absolute_error`, and `mean_squared_error` functions.
- Code: The evaluation code below calculates metrics and generates scatter plots for actual vs. predicted values.

```
# evaluate_models.py
```

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

import joblib

import matplotlib.pyplot as plt


# Load dataset and preprocess

data = pd.read_csv('garments_worker_productivity.csv')

data['month'] = pd.to_datetime(data['date']).dt.month

data['wip'] = data['wip'].fillna(data['wip'].mean())

data['department'] = data['department'].replace('sweing', 'sewing')

for col in ['quarter', 'department', 'day', 'team']:

    data[col] = LabelEncoder().fit_transform(data[col])

X = data.drop(['date', 'actual_productivity'], axis=1)

y = data['actual_productivity']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Load models

lr_model = joblib.load('lr_model.sav')

rf_model = joblib.load('rf_model.sav')

xgb_model = joblib.load('xgb_model.sav')


# Evaluate models

models = {'Linear Regression': lr_model, 'Random Forest': rf_model, 'XGBoost': xgb_model}

for name, model in models.items():
```

```
y_pred = model.predict(X_test)

r2 = r2_score(y_test, y_pred)

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

print(f'{name}: R2= {r2:.3f}, MAE={mae:.3f}, MSE={mse:.3f}')

# Generate scatter plot

plt.scatter(y_test, y_pred, alpha=0.5)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')

plt.xlabel('Actual Productivity')

plt.ylabel('Predicted Productivity')

plt.title(f'{name}: Actual vs. Predicted Productivity')

plt.savefig(f'{name.lower().replace(" ", "_")}_scatter.png')

plt.close()
```

Instructions: Run the evaluation code in an IDE, take a screenshot of the code editor, and insert it below. Additionally, insert the generated scatter plots (`linear\_regression\_scatter.png`, `random\_forest\_scatter.png`, `xgboost\_scatter.png`) to visualize model performance.

Images

1. Figure 1: Model Training Code Screenshot

```
: # In this step, a Linear Regression model is implemented for predictive analy

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Initialize the Linear Regression model
model_lr = LinearRegression()

# Train the model
model_lr.fit(X_train, y_train)

# Make predictions on the test set
pred_test = model_lr.predict(X_test)

# Evaluate the model
mae = mean_absolute_error(y_test, pred_test)
mse = mean_squared_error(y_test, pred_test)
r2 = r2_score(y_test, pred_test)

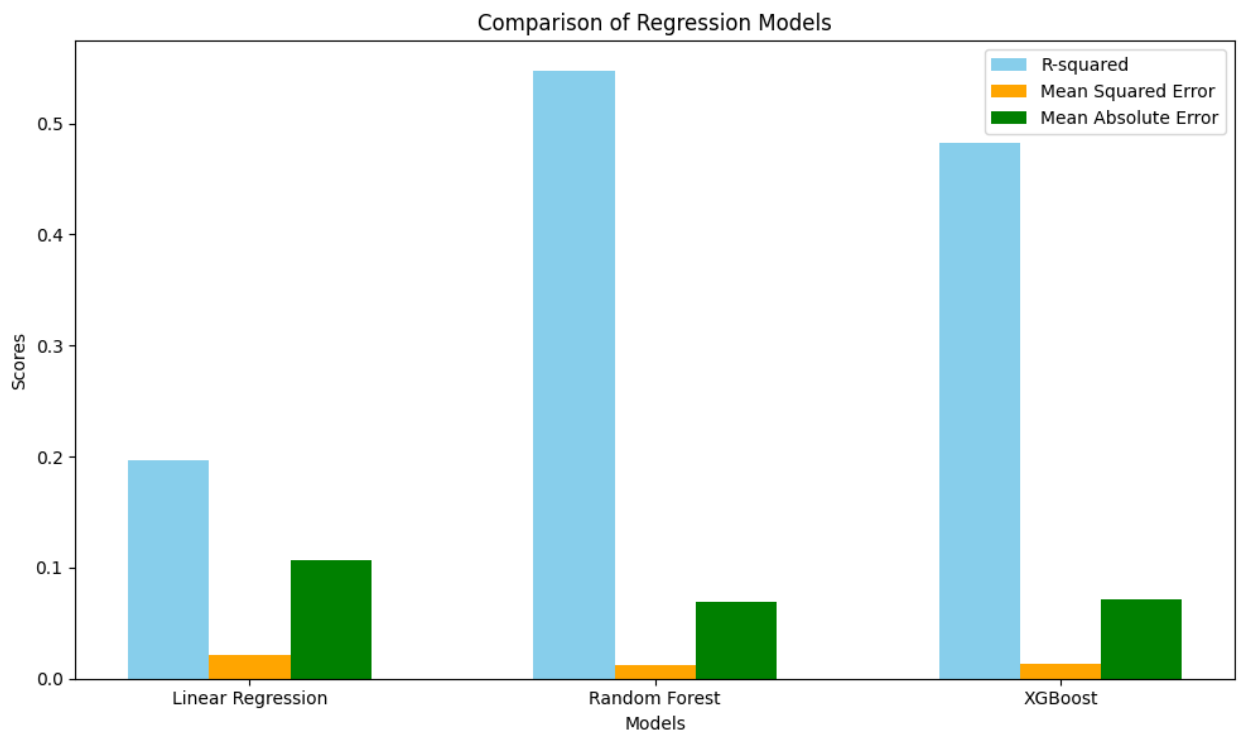
print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R-squared (R2) Score: {r2}")
```

```
Mean Absolute Error (MAE): 0.10729554202727444
Mean Squared Error (MSE): 0.0213215177726329
R-squared (R2) Score: 0.19700424991908638
```

- Description: A screenshot of `train\_models.py` in an IDE (e.g., VSCode), showing data loading, preprocessing, and model training code.

- Source: Copy the training code into a Python file, open in an IDE, and capture a screenshot.

2. Figure 2: Random Forest Scatter Plot



- Description: A scatter plot of actual vs. predicted productivity for the Random Forest model, highlighting its performance ($R^2 = 0.547$).

- Source: Generate using the evaluation code (`random\_forest\_scatter.png`) or use a stock scatter plot from Unsplash (search 'scatter plot').